Segundo Parcial

Inteligencia Artificial · Ingeniería de Sistemas · Período 2026-02 · Docente: Jhon James Cano Sánchez

Campo	Detalle
Asignatura	Inteligencia Artificial
Fecha	Lunes 26 de octubre de 2026
Cierre de la entrega	**10:30 p. m.** en AVACO.
Modalidad	Individual o en parejas (máximo 2 estudiantes)
Ponderación	10 % del segundo corte (30 %)
Herramientas permitidas	IA, internet, documentación y todo el material del curso (guías y código de las clases 7, 8 y 9)
Entrega	Enlace del repositorio público de GitHub en la plataforma AVACO antes de terminar la hora de clase

1. Visión General

El parcial evalúa la capacidad del estudiante para:

- Configurar un entorno de desarrollo desde cero (Python + Docker + WSL).
- Detectar y corregir problemas de calidad en datos de la región usando **Pandas**.
- Visualizar relaciones entre variables con **Seaborn**.
- Construir **desde cero, con NumPy**, un perceptrón con función sigmoide y descenso del gradiente (Clase 9).
- Interpretar lo que el modelo aprendió y comunicarlo en un README de GitHub.

Característica: cada grupo recibe un archivo CSV diferente. Las tareas son las mismas, pero las respuestas dependen de los datos específicos de cada archivo y de las decisiones que tome el grupo.

Instrucciones

1. Usar el archivo asignado para cada grupo en la tabla **Grupos y Archivo de Datos Asignado**.
2. Descargar de la plataforma AVACO el archivo `grupo_XX.csv` correspondiente al número asignado.
3. Se pueden usar todas las herramientas disponibles, incluida la IA. Las respuestas de interpretación deben referirse a los números **del propio archivo**: respuestas genéricas no reciben puntaje.
4. Solo se permite conversar o intercambiar información con el compañero de parcial. Si el parcial es individual, no se puede consultar a otro grupo o persona.

5. **El modelo debe construirse desde cero con NumPy**, como en la Clase 9. No se permite usar Scikit-Learn ni ninguna otra librería de Machine Learning para el perceptrón; si se usa, la tarea 5 se califica con 0.0.

Grupos y Archivo de Datos Asignado

Grupo	Integrante 1	Integrante 2	Archivo
01	Manuela Aguirre Toro	Alejandra Díaz Velásquez	`grupo_01.csv`
02	Brandon Cortes Giraldo	Johan Sttive Linares Barragán	`grupo_02.csv`
03	Davison Cruz López	Juan Diego Zapata Arbeláez	`grupo_03.csv`
04	Damián Correa Tusarma	Saray Foronda Restrepo	`grupo_04.csv`
05	Andrés Felipe Mejía Suaza	Miguel Ángel Ramírez Gutiérrez	`grupo_05.csv`
06	Jhon Esteban Molina Echavarría	Heiber Lozano Mercado	`grupo_06.csv`
07	Juan Esteban Villamil Echavarría	Santiago Hidalgo López	`grupo_07.csv`
08	Gustavo Adolfo Piedrahita López	Individual	`grupo_08.csv`
09	Cristian David Zuleta Restrepo	Individual	`grupo_09.csv`
10	Tatiana Vanessa Alvis	Individual	`grupo_10.csv`
11	Sofía Díaz Matallán	Individual	`grupo_11.csv`
12	Mónica Alejandra Parra Quintero	Individual	`grupo_12.csv`
13	Juan Sebastián Jiménez Cardona	Individual	`grupo_13.csv`
14	Manuel Fernando Montoya Bohórquez	Individual	`grupo_14.csv`

> **Estudiantes individuales:** Quien trabaja solo en el proyecto presenta el parcial de forma individual. Si dos estudiantes individuales deciden hacer el parcial juntos, deben **indicarlo en la entrega**: escribir los dos nombres al inicio del README y en el comentario de la entrega en AVACO, y trabajar con el archivo del **número de grupo menor** de los dos. Ambos cargan el mismo enlace.

Cada grupo trabaja **únicamente** con su archivo. El nombre del repositorio debe llevar el número del grupo: `parcial2-ia-grupoXX` (por ejemplo, `parcial2-ia-grupo04`).

2. Enunciado

Cartago, Valle del Cauca, enfrenta desafíos en agricultura, movilidad, salud, educación, medio ambiente, comercio y gobierno. El grupo ha sido contratado como consultor de IA por una entidad local que quiere **anticipar una situación de sí/no** (por ejemplo, si una cosecha será buena o si una cita médica se perderá) a partir de dos variables numéricas. La entidad entregó un archivo exportado de su sistema de información, **sin depurar**.

> **Atención:** Los datos fueron exportados tal como estaban en el sistema. Antes de entrenar cualquier modelo es responsabilidad del consultor revisar su calidad. Un modelo entrenado con datos sucios toma malas decisiones.

Archivo asignado

Cada archivo contiene: una columna identificadora, una columna categórica (zona, municipio, programa, etc.), **dos columnas numéricas** que serán las entradas del perceptrón y, en la última posición, la **columna objetivo** (1 o 0). El significado de las columnas de cada archivo está en el **Anexo A**.

3. Tareas

Tarea 1. Configuración del entorno (0.5)

- Crear un repositorio público en GitHub llamado `parcial2-ia-grupoXX` (XX = número de grupo).
- Desde la carpeta del curso en WSL, crear la carpeta `parcial2` con las subcarpetas `src` y `data`, y copiar allí el CSV asignado.
- Crear desde cero `Dockerfile`, `docker-compose.yml` y `requirements.txt` con Python 3.12, NumPy, Pandas, Matplotlib y Seaborn.
- En `docker-compose.yml` usar `container_name: parcial2_ia` para no chocar con el contenedor `python_ia` de las clases.
- Documentar en el README los comandos para reproducir el entorno.

Tarea 2. Carga e inspección de datos (0.4)

- Crear el script `src/parcial2.py` que cargue el CSV con Pandas.
- Mostrar en consola: las primeras 5 filas, la forma del DataFrame (`shape`), los tipos de datos (`info()` o `dtypes`), la cantidad de nulos por columna y la cantidad de filas duplicadas.
- **Pregunta clave:** ¿Alguna columna numérica aparece con un tipo de dato que no debería tener? ¿Por qué ocurre?

Tarea 3. Calidad y limpieza de datos con Pandas (1.2)

- Identificar y corregir **todos** los problemas de calidad del archivo (para el puntaje completo se esperan al menos 3). Algunos tipos posibles: valores faltantes, filas duplicadas, filas de resumen, valores con unidades equivocadas, valores imposibles, texto dentro de columnas numéricas, categorías escritas de formas distintas.
- Para cada problema documentar en el README:
  - ¿Cómo se detectó? (código o salida de consola que lo evidencia).
  - ¿Qué decisión se tomó? (eliminar, corregir, convertir, unificar).
  - ¿Por qué se tomó esa decisión?
- Informar el número de filas **antes y después** de la limpieza y cuántos registros quedaron de cada clase (0 y 1).
- Guardar el resultado en `data/grupo_XX_limpio.csv`.
- **Pregunta clave:** ¿Qué valor le pareció más sospechoso? Calcular la media de esa columna **antes y después** de corregirlo y explicar cómo habría afectado el escalado del perceptrón si no se hubiera corregido.

Tarea 4. Visualización con Seaborn (0.6)

- Gráfico 1: dispersión de las dos variables numéricas coloreada por la columna objetivo (`sns.scatterplot(..., hue=...)`).
- Gráfico 2: un gráfico que compare una variable numérica entre las clases 0 y 1 (`sns.boxplot`) **o** la columna categórica frente al objetivo (`sns.countplot(..., hue=...)`).
- Guardar ambos con `plt.savefig()` y escribir en el README qué muestra cada uno y qué conclusión se extrae.

Tarea 5. Perceptrón desde cero con NumPy (1.3)

- Separar `X` (las dos columnas numéricas) e `y` (la columna objetivo) como arreglos de NumPy.
- Estandarizar `X` (media 0 y desviación 1 por columna) y guardar la media y la desviación.
- Implementar las funciones: sigmoide, predicción de probabilidad (`X @ w + b`), pérdida logarítmica y exactitud.
- Entrenar con descenso del gradiente durante al menos 300 épocas, registrando la pérdida y la exactitud de cada época.

- Mostrar en consola: pesos iniciales y finales (`w1`, `w2`, `b`), pérdida inicial y final, y exactitud final.
- Generar `curva_aprendizaje.png` (pérdida por época) y `frontera_decision.png` (puntos de las dos clases y la línea de la frontera en las **unidades originales**).

Tarea 6. Interpretación de resultados (0.7)

1. ¿Qué significa el signo de `w1` y de `w2` en el contexto del problema? ¿Tiene sentido para Cartago?
2. Según la frontera de decisión, ¿los datos son aproximadamente linealmente separables? ¿Cuántos registros quedan del lado equivocado?
3. Inventar dos registros nuevos realistas, uno que el grupo espera que sea 1 y otro que espera que sea 0. Escalarlos con la media y la desviación del entrenamiento, calcular su probabilidad con el modelo y comentar si el resultado coincide con lo esperado.
4. ¿Qué recomendación concreta le haría a la entidad local? Justificar con al menos 2 números obtenidos en el parcial.
5. ¿Qué limitaciones tiene este modelo para tomar decisiones reales?

Tarea 7. Repositorio y documentación (0.3)

- Repositorio con: `Dockerfile`, `docker-compose.yml`, `requirements.txt`, `src/parcial2.py`, el CSV original, el CSV limpio, los 4 gráficos PNG y el `README.md` con todas las respuestas.
- Al menos 3 commits durante el parcial (no un único commit al final).
- Cargar el enlace del repositorio en AVACO.

4. Distribución Sugerida del Tiempo

Tarea	Tiempo sugerido
1. Entorno	10 min
2. Carga e inspección	10 min
3. Limpieza	20 min
4. Visualización	10 min
5. Perceptrón	20 min
6 y 7. Interpretación y repositorio	20 min

> **Consejo:** Hacer commit al terminar cada tarea. Si algo falla al final, lo que ya está en GitHub cuenta.

5. Rúbrica de Evaluación

Tarea	Completo	Parcial	Puntaje máximo
1. Configuración del entorno	Contenedor funcional creado desde cero y comandos documentados.	Entorno funcional sin documentar o con archivos copiados sin ajustar.	0.5
2. Carga e inspección	Todas las salidas pedidas y respuesta a la pregunta clave.	Faltan salidas o la pregunta clave.	0.4
3. Calidad y limpieza	Detecta y corrige todos los problemas, justifica cada decisión, informa filas y clases, responde la pregunta clave con números.	Detecta menos problemas o no justifica las decisiones.	1.2
4. Visualización con Seaborn	Dos gráficos correctos, con títulos, ejes e interpretación.	Gráficos sin interpretación o incompletos.	0.6
5. Perceptrón desde cero	Escalado, sigmoide, pérdida y descenso del gradiente implementados con NumPy; la pérdida baja; ambos gráficos generados.	El modelo entrena pero falta algún componente o gráfico.	1.3
6. Interpretación	Respuestas específicas con los números del propio archivo.	Respuestas genéricas o incompletas.	0.7
7. Repositorio y documentación	Todos los archivos, ≥3 commits y README organizado.	Faltan archivos o hay un solo commit.	0.3
		Total	**5.0**

Anexo A. Descripción de los Archivos

Archivo	Tema	Entradas (x1, x2)	Objetivo (y)
`grupo_01.csv`	Café – fincas cafeteras	`altitud_msnm`, `lluvia_mm_mes`	`cosecha_buena`: 1 = cosecha buena, 0 = cosecha deficiente
`grupo_02.csv`	Caña de azúcar – lotes	`horas_riego_semana`, `fertilizante_kg_ha`	`rendimiento_alto`: 1 = rendimiento alto, 0 = rendimiento bajo
`grupo_03.csv`	Turismo – hoteles y hostales	`precio_noche_miles`, `calificacion`	`ocupacion_alta`: 1 = ocupación alta, 0 = ocupación baja
`grupo_04.csv`	Movilidad – tramos viales	`velocidad_promedio_kmh`, `flujo_vehiculos_hora`	`accidente_reportado`: 1 = hubo accidente reportado, 0 = no
`grupo_05.csv`	Calidad del aire – mediciones	`pm25_ugm3`, `viento_m_s`	`alerta_calidad`: 1 = se emitió alerta de calidad del aire, 0 = no
`grupo_06.csv`	Consumo de agua – hogares	`consumo_m3_mes`, `personas_hogar`	`consumo_excesivo`: 1 = consumo excesivo, 0 = consumo normal
`grupo_07.csv`	Residuos – rutas de recolección	`toneladas_recogidas`, `distancia_km`	`ruta_retrasada`: 1 = la ruta terminó con retraso, 0 = a tiempo
`grupo_08.csv`	Salud – citas médicas	`edad_anios`, `dias_anticipacion`	`cita_perdida`: 1 = el paciente no asistió, 0 = asistió
`grupo_09.csv`	Educación – estudiantes	`asistencia_pct`, `promedio_notas`	`desercion`: 1 = desertó, 0 = continúa
`grupo_10.csv`	Comercio – clientes de tienda	`visitas_mes`, `ticket_promedio_miles`	`cliente_recurrente`: 1 = cliente recurrente, 0 = ocasional
`grupo_11.csv`	PQRS – solicitudes ciudadanas	`dias_respuesta`, `reiteraciones`	`escalada`: 1 = la PQRS fue escalada a control interno, 0 = no
`grupo_12.csv`	Clima – riesgo de incendio forestal	`temperatura_c`, `humedad_relativa_pct`	`riesgo_incendio`: 1 = riesgo alto de incendio, 0 = riesgo bajo

Unidades esperadas: altitud en metros sobre el nivel del mar; lluvia en mm por mes; precios y tickets en miles de pesos; velocidad en km/h; flujo en vehículos por hora; PM2.5 en µg/m^3; viento en m/s; consumo en m^3 por mes; residuos en toneladas; distancia en km; edad en años; asistencia y humedad en porcentaje (0–100); promedio de notas en escala 0–5; temperatura en °C.

Anexo B. Recordatorio de Comandos

Comandos de referencia

```
# En WSL, desde la carpeta del curso
mkdir -p parcial2/src parcial2/data
cd parcial2
docker ps                           # si python_ia está encendido: docker stop python_ia
docker-compose up -d --build
docker exec -it parcial2_ia bash

# Dentro del contenedor
cd src
python parcial2.py
exit

# En WSL, para versionar (dentro de la carpeta parcial2)
git init
git add .
git commit -m "Parcial 2: entorno y carga de datos"
git branch -M main
git remote add origin <URL-del-repositorio>
git push -u origin main
```

Si la carpeta `parcial2` es el repositorio, el `docker-compose.yml` y el `Dockerfile` deben quedar en su raíz.